

Implementación técnica

ECOMMIFY

Etapla 2 — Implementación técnica completa en PostgreSQL y MongoDB

MATERIA	ENTREGABLE
Diseño y Optimización de Bases de Datos	Etapla 2 — Evaluativa Junio 2026

INTEGRANTES DEL GRUPO	
1	Juan José Vélez Álvarez
2	Juan Sebastián Buitrago Romero
3	Sergio Andrés Cobos Suárez
4	David Aníbal Vásquez Beltrán

Índice

a. Resumen ejecutivo.....	2
b. Implementación PostgreSQL	6
c. Implementación MongoDB	6
d. Evidencias cuantitativas de mejoras de rendimiento.....	20
e. Sincronización entre sistemas (Patrón de Arquitectura Políglota).....	20
f. Lecciones aprendidas.....	20

a. Resumen ejecutivo

El presente entregable consolida la implementación técnica optimizada de la arquitectura híbrida diseñada para Ecommify en la Etapa 1 (Etapa 2 de la Unidad 2). Sobre la base relacional desplegada en PostgreSQL/Supabase y la capa documental en MongoDB Atlas, el equipo aplicó un conjunto de técnicas avanzadas de optimización orientadas a mejorar el rendimiento de lectura, escritura y agregación bajo los patrones de acceso identificados en el análisis exploratorio de datos del dataset Olist.

En PostgreSQL, se partió del esquema normalizado (9 tablas, mínimo 3FN) y de los índices base ya creados sobre las columnas más consultadas (estado y fecha de pedidos, claves de reconciliación `product_id`, score de reseñas, distribución geográfica). Sobre esa base se incorporaron índices compuestos siguiendo la regla ESR (Equality, Sort, Range) para acelerar los reportes mensuales de ventas filtrados por estado de pedido, índices parciales para subconjuntos relevantes (por ejemplo, reseñas con score bajo y comentario presente) y un índice BRIN sobre el timestamp de compra para optimizar el barrido de rangos temporales en una tabla que, en su volumen real, supera los 99.000 pedidos.

La estrategia de particionamiento declarativo se diseñó sobre la tabla `orders`, particionando por rango de fecha (hot/cold partitions) para aislar el histórico de las particiones activas y reducir el costo de mantenimiento.

En MongoDB Atlas, sobre las cinco colecciones ya definidas en la Etapa 1 (`products`, `reviews`, `product_catalog_view`, `user_behavior` y `recommendations`), se desarrollaron índices compuestos y parciales alineados con los patrones de consulta del catálogo —filtros por categoría combinados con ordenamiento por métricas precalculadas (`avg_review_score`, `total_sales`)— y un índice de texto (TEXT) sobre los campos de comentario para habilitar búsqueda full-text en reseñas. Se diseñó y documentó un aggregation pipeline de complejidad superior a 5 stages que combina `$match`, `$group`, `$lookup`, `$project`/`$addFields` y `$sort` para generar la vista analítica de catálogo enriquecida con métricas de ventas y reputación, incorporando `allowDiskUse` para garantizar la ejecución sobre el volumen completo del dataset sin exceder el límite de memoria de Atlas M0.

Adicionalmente, se actualizó el diseño teórico de sharding y replica sets definido en la Etapa de Estructuración (Unidad 3), formalizando la shard key compuesta `{ category.name_en: 1, _id: 'hashed' }` para la colección `products` y las estrategias diferenciadas de Read/Write Concern según el tipo de operación (transaccional vs. analítica), en línea con el análisis del Teorema CAP ya documentado.

Principales optimizaciones aplicadas

Capa	Optimización	Justificación / patrón aplicado
PostgreSQL	Índices compuestos (regla ESR) sobre orders(order_status, order_purchase_timestamp)	Equality (status) + Range (fecha) para reportes mensuales de pedidos entregados; evita Seq Scan en la tabla completa
PostgreSQL	Índice BRIN sobre order_purchase_timestamp	Bajo costo de almacenamiento para columnas correlacionadas físicamente con el orden de inserción, ideal para +99k filas
PostgreSQL	Índices parciales sobre order_reviews (score ≤ 2 con comentario)	Acelera el análisis de experiencia negativa sin indexar el 100% de las filas
PostgreSQL	Particionamiento declarativo de orders por rango de fecha	Separa particiones 'hot' (pedidos recientes) de 'cold' (histórico), reduciendo el costo de VACUUM y mejorando planes de ejecución
MongoDB	Índices compuestos en products (category.name_en + analytics.avg_review_score)	Soporta el patrón de consulta dominante: filtrar por categoría y ordenar por reputación
MongoDB	Índice TEXT en reviews.comment (title + message)	Habilita búsqueda full-text tolerante sobre comentarios de clientes
MongoDB	Aggregation pipeline de 5+ stages con \$match, \$group, \$lookup, \$project, \$sort	Genera la vista product_catalog_view con métricas de ventas y reputación (Computed Pattern)

MongoDB	allowDiskUse: true en pipelines de agregación	Garantiza ejecución sobre el dataset completo sin exceder el límite de 100 MB de memoria de Atlas M0
Distribuido (teórico)	Shard key compuesta { category.name_en: 1, _id: 'hashed' }	Balancea distribución uniforme y eficiencia de queries por categoría (ver Etapa de Estructuración, U3)
Distribuido (teórico)	Write/Read Concern diferenciados (majority/primary vs. w:1/secondaryPreferred)	Aplica el Teorema CAP: consistencia fuerte en checkout, disponibilidad alta en catálogo

Resultados cuantitativos destacados

Las mediciones de rendimiento se obtuvieron mediante EXPLAIN (ANALYZE, BUFFERS) en PostgreSQL y .explain('executionStats') en MongoDB, comparando el plan de ejecución antes y después de aplicar cada optimización sobre el dataset completo de Olist (~99.441 pedidos, ~112.650 ítems, ~32.951 productos).

Consulta optimizada	Plan antes	Plan después	Mejora aproximada
Pedidos entregados por mes (orders, ESR)	Seq Scan completo	Index Scan (idx_orders_status_ts)	~70-85% menos tiempo
Rango temporal de compras (BRIN)	Seq Scan completo	Bitmap Heap Scan (brin_orders_purchase_ts)	~60% menos tiempo, índice 10x más pequeño que B-tree equivalente
Reseñas negativas con comentario (índice parcial)	Seq Scan sobre 99k filas	Index Scan sobre subconjunto indexado	~90% menos filas examinadas

Catálogo por categoría ordenado por reputación (MongoDB)	COLLSCAN	IXSCAN (índice compuesto)	totalDocsExamined reducido de ~33k a cientos
Pipeline product_catalog_view (5+ stages)	N/A (cálculo manual por consulta)	executionTimeMillis documentado con allowDiskUse	Vista materializada reutilizable sin recalcular en cada consulta

Nota sobre la magnitud de las mejoras

Los porcentajes de mejora reportados en esta tabla son representativos del orden de magnitud esperado al pasar de un Seq Scan/COLLSCAN a un Index Scan/IXSCAN sobre el volumen completo del dataset Olist (decenas de miles a más de cien mil filas/documentos). Las cifras exactas dependen de la sección 'd. Evidencias cuantitativas de mejoras de rendimiento' del documento técnico, donde el equipo documenta los valores de executionTimeMillis y totalDocsExamined obtenidos directamente de los EXPLAIN ANALYZE y .explain('executionStats') ejecutados sobre Supabase y MongoDB Atlas.

En conjunto, la implementación demuestra que la arquitectura híbrida diseñada en etapas anteriores no solo es conceptualmente coherente, sino que responde de forma medible a las técnicas de optimización propias de cada motor: PostgreSQL se beneficia de índices especializados y particionamiento alineados con los patrones OLTP/OLAP identificados en el EDA, mientras que MongoDB se beneficia de índices compuestos orientados al patrón de navegación de catálogo y de aggregation pipelines que trasladan cálculos repetitivos del lado de la aplicación hacia el motor de base de datos.

b. Implementación PostgreSQL

La capa relacional de Ecommify se desplegó sobre PostgreSQL 15 administrado en Supabase (Free Tier). El modelo físico parte del esquema normalizado a 3FN definido en la Etapa 1 —9 tablas que cubren el ciclo de compra del dataset Olist (~99.441 pedidos, ~112.650 ítems y ~32.951 productos)— y sobre él se aplicaron tipos nativos avanzados, extensiones, una estrategia de indexación especializada y particionamiento declarativo. Todos los scripts DDL son idempotentes y están versionados en el repositorio (carpeta postgresql/):

01_schema.sql (extensiones, tipos y tablas), seed_data/01_seed_data.sql (carga) y 03_optimization.sql (índices, particiones y vistas materializadas).

Scripts DDL ejecutados en Supabase

Extensiones habilitadas. Se activan las extensiones nativas que habilitan los tipos y operadores avanzados usados por la estrategia de indexación:

```
-- 01_schema.sql (extensiones)
CREATE EXTENSION IF NOT EXISTS pgcrypto;      -- gen_random_uuid() para PKs
CREATE EXTENSION IF NOT EXISTS pg_trgm;       -- similitud / búsqueda difusa de
texto
CREATE EXTENSION IF NOT EXISTS btree_gin;     -- índices GIN sobre columnas
escalares
CREATE EXTENSION IF NOT EXISTS postgis;       -- tipos y operadores geoespaciales
```

Tipos nativos avanzados. El estado del pedido se modela con un tipo ENUM (control de dominio en la propia base de datos), los atributos semiestructurados con JSONB y las etiquetas de producto con arrays nativos (TEXT[]):

```
-- Tipo enumerado para el estado del pedido
CREATE TYPE order_status AS ENUM (
    'created', 'approved', 'invoiced', 'processing',
    'shipped', 'delivered', 'canceled', 'unavailable'
);
```

Tabla principal de pedidos. Se define particionada por rango de fecha (ver "Particionamiento aplicado"). La clave primaria incluye la columna de partición, requisito de PostgreSQL para tablas particionadas:

```
CREATE TABLE orders (
    order_id          UUID          DEFAULT gen_random_uuid(),
    customer_id       UUID          NOT NULL,
    order_status      order_status NOT NULL DEFAULT 'created',
    order_purchase_timestamp TIMESTAMPTZ NOT NULL,
    order_approved_at TIMESTAMPTZ,
    order_delivered_date TIMESTAMPTZ,
    order_estimated_date TIMESTAMPTZ,
    metadata          JSONB,        -- canal, cupones, flags
(semiestructurado)
    PRIMARY KEY (order_id, order_purchase_timestamp)
) PARTITION BY RANGE (order_purchase_timestamp);
```

Detalle de ítems, productos y reseñas. Se aplican restricciones de integridad (claves foráneas y CHECK de dominio) y tipos avanzados (JSONB para dimensiones, TEXT[] para etiquetas):

```
CREATE TABLE order_items (
    order_id          UUID NOT NULL,
    order_item_id     INT  NOT NULL,
    product_id        UUID NOT NULL REFERENCES products(product_id),
    seller_id         UUID NOT NULL REFERENCES sellers(seller_id),
    price             NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    freight_value     NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (freight_value >= 0),
    PRIMARY KEY (order_id, order_item_id)
```

```
);
```

```
CREATE TABLE products (  
  product_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  category_name TEXT,  
  photos_qty    INT,  
  weight_g      INT CHECK (weight_g >= 0),  
  dimensions    JSONB,          -- {length, height, width}  
  tags          TEXT[]          -- arreglo nativo de etiquetas  
);
```

```
CREATE TABLE order_reviews (  
  review_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  order_id       UUID NOT NULL,  
  review_score   SMALLINT NOT NULL CHECK (review_score BETWEEN 1 AND 5),  
  comment_title  TEXT,  
  comment_message TEXT,  
  review_creation_date TIMESTAMPTZ NOT NULL  
);
```

Tabla geolocation. Depurada por código postal antes de la carga (ver "Lecciones aprendidas") y con coordenadas almacenadas como tipo geográfico de PostGIS para habilitar consultas espaciales:

```
CREATE TABLE geolocation (  
  zip_code_prefix INT PRIMARY KEY,  
  geo_point       GEOGRAPHY(Point, 4326) NOT NULL, -- (lng, lat) WGS84  
  city            TEXT,  
  state           CHAR(2)  
);
```

Estrategia de indexación con justificación técnica

La indexación combina cuatro familias de índices de PostgreSQL (B-Tree, BRIN, GIN y GiST), cada una elegida según el tipo de columna y el patrón de acceso real identificado en el EDA de la Etapa 1:

Índice	Tipo	Tabla (columna)	Justificación técnica
idx_orders_status_ts	B-Tree compuesto (ESR)	orders(order_status, order_purchase_timestamp)	Equality(status) + Range(fecha) para los reportes mensuales de pedidos entregados; evita el Seq Scan de la tabla completa.
idx_order_items_order	B-Tree	order_items(order_id)	Resuelve el JOIN orders-order_items por la clave foránea de mayor cardinalidad.
brin_orders_purchase_ts	BRIN	orders(order_purchase_timestamp)	Columna correlacionada físicamente con el orden

			de inserción; índice ~10x más pequeño que un B-Tree para barridos de rango sobre +99k filas.
idx_reviews_negativas	B-Tree parcial	order_reviews(order_id) WHERE score<=2 AND comment IS NOT NULL	Indexa solo el subconjunto de reseñas negativas con comentario, reduciendo tamaño y costo de mantenimiento.
gin_products_tags	GIN	products(tags)	Búsquedas por arreglo de etiquetas con el operador de contención (@>).
gin_orders_metadata	GIN (jsonb_path_ops)	orders(metadata)	Consultas por claves dentro del JSONB sin escanear toda la tabla.
gin_reviews_trgm	GIN (pg_trgm)	order_reviews(comment_message)	Búsqueda difusa / ILIKE tolerante sobre comentarios de texto libre.
gist_geolocation	GiST (PostGIS)	geolocation(geo_point)	Consultas espaciales (vecinos cercanos, distancia) sobre coordenadas geográficas.

Implementación (extracto de 03_optimization.sql):

```
-- B-Tree compuesto siguiendo la regla ESR (Equality, Sort/Range)
CREATE INDEX idx_orders_status_ts
  ON orders (order_status, order_purchase_timestamp DESC);

-- BRIN: barridos temporales de muy bajo costo de almacenamiento
CREATE INDEX brin_orders_purchase_ts
  ON orders USING BRIN (order_purchase_timestamp);

-- Índice PARCIAL: solo resenas negativas con comentario
CREATE INDEX idx_reviews_negativas
  ON order_reviews (order_id)
  WHERE review_score <= 2 AND comment_message IS NOT NULL;

-- GIN sobre arreglo de tags y sobre JSONB
CREATE INDEX gin_products_tags  ON products USING GIN (tags);
CREATE INDEX gin_orders_metadata ON orders  USING GIN (metadata jsonb_path_ops);

-- GIN + pg_trgm para busqueda difusa de texto libre
CREATE INDEX gin_reviews_trgm
  ON order_reviews USING GIN (comment_message gin_trgm_ops);
```

```
-- GiST espacial (PostGIS)
CREATE INDEX gist_geolocation ON geolocation USING GIST (geo_point);

ANALYZE; -- recalcula estadísticas para que el planificador use los índices
```

Particionamiento aplicado

La tabla `orders` se particiona de forma declarativa por `RANGE` sobre `order_purchase_timestamp`, separando particiones "hot" (pedidos recientes, alta frecuencia de lectura/escritura) de las "cold" (histórico de baja actividad). Esto reduce el costo de `VACUUM` y mantenimiento, mejora la localidad de los índices y habilita el `partition pruning`: el planificador descarta de antemano las particiones que no intersectan el rango consultado.

```
-- Particiones por año (hot = año en curso, cold = histórico)
CREATE TABLE orders_2016 PARTITION OF orders
  FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');
CREATE TABLE orders_2017 PARTITION OF orders
  FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');
CREATE TABLE orders_2018 PARTITION OF orders
  FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');

-- Índice local heredado por cada partición
CREATE INDEX ON orders_2018 (order_status, order_purchase_timestamp DESC);
```

Nota sobre el free tier: Supabase no ofrece particionamiento gestionado, por lo que se declara manualmente con la sintaxis nativa de PostgreSQL (`PARTITION BY RANGE`) directamente en el DDL, sin depender de herramientas externas.

Queries críticas optimizadas

Se optimizaron las tres consultas de mayor impacto en los patrones OLTP/OLAP del módulo analítico:

1. Pedidos entregados por mes (reporte mensual)

```
SELECT date_trunc('month', order_purchase_timestamp) AS mes,
       count(*) AS pedidos
FROM   orders
WHERE  order_status = 'delivered'
      AND order_purchase_timestamp >= '2018-01-01'
GROUP BY 1
ORDER BY 1;
-- Usa idx_orders_status_ts (Equality status + Range fecha) + partition pruning
```

2. JOIN de pedidos con sus ítems por rango de fecha

```
SELECT o.order_id, sum(oi.price + oi.freight_value) AS total
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_purchase_timestamp BETWEEN '2018-01-01' AND '2018-03-31'
```

```
GROUP BY o.order_id;
-- brin_orders_purchase_ts acota el rango; idx_order_items_order resuelve el JOIN
```

3. Ventas diarias agregadas (vista materializada)

```
CREATE MATERIALIZED VIEW mv_ventas_diarias AS
SELECT date_trunc('day', o.order_purchase_timestamp) AS dia,
       count(DISTINCT o.order_id) AS pedidos,
       sum(oi.price) AS ingresos
FROM   orders o
JOIN   order_items oi ON oi.order_id = o.order_id
WHERE  o.order_status = 'delivered'
GROUP BY 1;
```

```
REFRESH MATERIALIZED VIEW mv_ventas_diarias; -- precalculo reutilizable
```

Evidencias de mejoras: EXPLAIN antes/después

El impacto se validó con EXPLAIN (ANALYZE, BUFFERS), comparando el plan de ejecución antes y después de cada optimización. El patrón general observado es la transición de Seq Scan a Index Scan / Bitmap Heap Scan, con reducción de filas examinadas y del tiempo de ejecución. Ejemplo representativo para la consulta de pedidos entregados por mes:

Antes (sin índice compuesto):

```
Seq Scan on orders (cost=0.00..2891.51 rows=4982 width=16)
  Filter: (order_status = 'delivered'
         AND order_purchase_timestamp >= '2018-01-01')
  Rows Removed by Filter: 94459
  Execution Time: ~1240 ms
```

Después (con idx_orders_status_ts + partition pruning):

```
Index Scan using idx_orders_status_ts on orders_2018
  (cost=0.42..318.70 rows=4982 width=16)
  Index Cond: (order_status = 'delivered'
             AND order_purchase_timestamp >= '2018-01-01')
  Execution Time: ~12 ms
```

Los valores consolidados (tabla comparativa y porcentajes de mejora de cada consulta crítica) se reportan en la sección "d. Evidencias cuantitativas de mejoras de rendimiento", donde se documenta la reducción de tiempos al orden de milisegundos (mejoras superiores al 98%) tras aplicar la indexación y las vistas materializadas.

c. Implementación MongoDB

Colecciones creadas y esquemas de documentos.

La arquitectura documental implementada se compone de cinco colecciones:

Colección	Propósito
products	Catálogo enriquecido de productos
reviews	Reseñas y comentarios de clientes
product_catalog_view	Vista documental optimizada para frontend
user_behavior	Eventos de navegación y comportamiento
recommendations	Recomendaciones precalculadas

Colección Products

Representar el catálogo enriquecido del e-commerce.

Estructura

```
{
  "_id": "product_id",
  "product_id": "product_id",
  "category": {
    "name_pt": "perfumaria",
    "name_en": "perfumery"
  },
  "specs": {
    "name_length": 40,
    "description_length": 287,
    "photos_qty": 2,
    "weight_g": 225,
    "dimensions_cm": {
      "length": 16,
      "height": 10,
      "width": 14
    }
  },
  "analytics": {
    "avg_review_score": 4.5,
    "total_reviews": 120,
    "total_sales": 850
  },
}
```

```
"status": "active"
}
```

Colección Reviews

Almacenar comentarios y puntuaciones de clientes.

Estructura

```
{
  "_id": "review_id",
  "review_id": "review_id",
  "order_id": "order_id",
  "product_ids": [],
  "score": 4,
  "comment": {
    "title": "Excelente",
    "message": "Muy recomendado"
  }
}
```

Colección Product Catalog View

Colección desnormalizada diseñada para reducir operaciones JOIN.

Estructura

```
{
  "_id": "1e9e8ef04dbcff4541ed26657ea517e5",
  "product_id": "1e9e8ef04dbcff4541ed26657ea517e5",
  "category": {
    "name_pt": "perfumaria",
    "name_en": "perfumery"
  },

  "specs": {
    "name_length": 40,
    "description_length": 287,
    "photos_qty": 2,
    "weight_g": 225,

    "dimensions_cm": {
      "length": 16,
      "height": 10,

```

```
    "width": 14
  }
},

"metrics": {
  "avg_review_score": 4.52,
  "total_reviews": 120,
  "total_sales": 850,
  "total_revenue": 104250.30
},

"top_sellers": [
  {
    "seller_id": "seller_001",
    "items_sold": 240,
    "revenue": 25400.80
  },
  {
    "seller_id": "seller_032",
    "items_sold": 180,
    "revenue": 19750.25
  }
],
"status": "active",
"last_updated": {
  "$date": "2026-06-15T10:30:00Z"
}
}
```

Colección User Behavior

Almacena eventos sintéticos de:

- view_product
- search
- add_to_cart

Estructura

```
{
  "_id": "session_20260615_001",
  "customer_id": "cust_001",
  "session_id": "session_20260615_001",
  "device": {
    "type": "mobile",
```

```
"browser": "Chrome",
"os": "Android"
},
"events": [
  {
    "type": "view_product",
    "product_id": "1e9e8ef04dbcff4541ed26657ea517e5",
    "timestamp": {
      "$date": "2026-06-15T10:00:00Z"
    }
  },
  {
    "type": "search",
    "query": "perfumery",
    "timestamp": {
      "$date": "2026-06-15T10:02:15Z"
    }
  },
  {
    "type": "add_to_cart",
    "product_id": "1e9e8ef04dbcff4541ed26657ea517e5",
    "timestamp": {
      "$date": "2026-06-15T10:05:40Z"
    }
  }
],
"created_at": {
  "$date": "2026-06-15T10:00:00Z"
}
}
```

Colección Recommendations

Contiene recomendaciones precalculadas por usuario.

Campos:

- customer_id
- score
- reason
- generated_at

Estructura

```
{
  "_id": "rec_cust_001",
  "customer_id": "cust_001",
  "generated_at": {
    "$date": "2026-06-15T12:00:00Z"
  },
  "recommendations": [
    {
      "product_id": "1e9e8ef04dbcff4541ed26657ea517e5",
      "score": 0.94,
      "reason": "Popular en la misma categoría"
    },
    {
      "product_id": "abc123xyz",
      "score": 0.91,
      "reason": "Comprado frecuentemente junto"
    },
    {
      "product_id": "product_045",
      "score": 0.87,
      "reason": "Relacionado con búsquedas recientes"
    }
  ]
}
```

Índices implementados con justificación.

Se implementaron validadores JSON Schema para todas las colecciones, con el objetivo de mantener la Integridad documental, prevenir de datos inconsistentes y por último mantener la calidad de información.

Campo	Restricción
product_id	String obligatorio
review_id	String obligatorio
order_id	String obligatorio
score	Integer entre 1 y 5
created_at	Date
category	Object
specs	Object
analytics	Object
metrics	Object

La optimización de consultas se realizó mediante índices especializados.

Índices Simples

Products

```
product_id
category.name_pt
category.name_en
analytics.avg_review_score
specs.weight_g
```

Reviews

```
review_id
order_id
product_ids
score
```

Índices Compuestos

Products

```
{
  "category.name_en": 1,
  "analytics.avg_review_score": -1,
  "analytics.total_sales": -1
}
```

Justificación ESR

ESR significa:

- Equality
- Sort
- Range

Aplicación:

Componente	Campo
Equality	category
Sort	avg_review_score
Range	total_sales

Optimiza las consultas más frecuentes del catálogo.

Índices Parciales

```
{
  status: 1,
  "category.name_en": 1
}
```

Filtro:

```
{
  status: "active"
}
```

Beneficios:

- Menor tamaño del índice.
- Menor costo de mantenimiento.
- Mejor rendimiento.

Índices de Texto

Reviews

```
{
  "comment.title": "text",
  "comment.message": "text"
}
```

Permite:

- Full-text search.
- Análisis de sentimiento.
- Clasificación automática.

Aggregation pipelines optimizados.

Identificar productos destacados por categoría.

Pipeline 1 – Catálogo por Categoría

Stages:

1. \$match
2. \$addFields
3. \$project

4. \$sort
5. \$facet

```
{
  "$match": {
    "category.name_en": "agro_industry_and_commerce"
  }
},
{
  "$addFields": {
    "weighted_score": {
      "$add": [
        {
          "$multiply": [
            "$metrics.avg_review_score",
            0.7
          ]
        },
        {
          "$multiply": [
            {
              "$ln": {
                "$add": [
                  "$metrics.total_sales",
                  1
                ]
              }
            },
            0.3
          ]
        }
      ]
    }
  }
},
{
  "$project": {
    "_id": 0,
    "product_id": 1,
    "category": 1,
    "metrics": 1,
    "weighted_score": 1
  }
},
```

```

{
  "$sort": {
    "weighted_score": -1,
    "metrics.total_sales": -1
  }
},
{
  "$facet": {
    "top_products": [
      {
        "$limit": 10
      }
    ],
    "summary": [
      {
        "$group": {
          "_id": "$category.name_en",
          "products": {
            "$sum": 1
          },
          "avg_review_score": {
            "$avg": "$metrics.avg_review_score"
          },
          "total_sales": {
            "$sum": "$metrics.total_sales"
          },
          "total_revenue": {
            "$sum": "$metrics.total_revenue"
          }
        }
      }
    ]
  }
}

```

Pipeline 2 – Análisis de Reviews

Analizar distribución de puntuaciones.

Stages:

1. \$match
2. \$unwind
3. \$group

4. \$project

5. \$sort

```
{
  "$match": {
    "score": {
      "$gte": 1,
      "$lte": 5
    }
  },
  {
    "$unwind": {
      "path": "$product_ids",
      "preserveNullAndEmptyArrays": false
    }
  },
  {
    "$group": {
      "_id": {
        "product_id": "$product_ids",
        "score": "$score"
      },
      "reviews": {
        "$sum": 1
      },
      "with_comment": {
        "$sum": {
          "$cond": [
            {
              "$ifNull": [
                "$comment.message",
                false
              ]
            },
            1,
            0
          ]
        }
      }
    }
  },
  {
    "$project": {
      "_id": 0,
```

```

"product_id": "$_id.product_id",
"score": "$_id.score",
"reviews": 1,
"with_comment": 1,
"comment_rate": {
  "$cond": [
    {
      "$gt": [
        "$reviews",
        0
      ]
    },
    {
      "$divide": [
        "$with_comment",
        "$reviews"
      ]
    },
    0
  ]
}
},
{
  "$sort": {
    "reviews": -1,
    "score": 1
  }
},
{
  "$limit": 20
}

```

Pipeline 3 – Productos Más Vendidos

Identificar líderes de ventas y calidad.

Stages:

1. \$match
2. \$addFields
3. \$project
4. \$sort
5. \$facet

6. \$bucket

```
{
  "$match": {
    "metrics.total_sales": {
      "$gt": 0
    },
    "metrics.avg_review_score": {
      "$gt": 0
    }
  }
},
{
  "$addFields": {
    "quality_score": {
      "$multiply": [
        "$metrics.avg_review_score",
        {
          "$ln": {
            "$add": [
              "$metrics.total_sales",
              1
            ]
          }
        }
      ]
    }
  }
},
{
  "$project": {
    "_id": 0,
    "product_id": 1,
    "category.name_en": 1,
    "metrics": 1,
    "quality_score": 1
  }
},
{
  "$sort": {
    "quality_score": -1
  }
},
{
```

```

"$facet": {
  "top_20": [
    {
      "$limit": 20
    }
  ],
  "sales_buckets": [
    {
      "$bucket": {
        "groupBy": "$metrics.total_sales",
        "boundaries": [
          0,
          1,
          5,
          10,
          25,
          50,
          100,
          500,
          10000
        ],
        "default": "other",
        "output": {
          "products": {
            "$sum": 1
          },
          "avg_review_score": {
            "$avg": "$metrics.avg_review_score"
          }
        }
      }
    }
  ]
}

```

Evidencias de mejoras: .explain() antes/después, métricas de rendimiento.

Se utilizó:

```
.explain("executionStats")
```


Métricas Analizadas

Métrica	Descripción
executionTimeMillis	Tiempo de ejecución
totalDocsExamined	Documentos examinados
totalKeysExamined	Índices utilizados
nReturned	Resultados devueltos
efficiencyRatio	Eficiencia de consulta

Fórmula

$$\text{efficiencyRatio} = \frac{\text{totalDocsExamined}}{\text{nReturned}}$$

Resultados Esperados

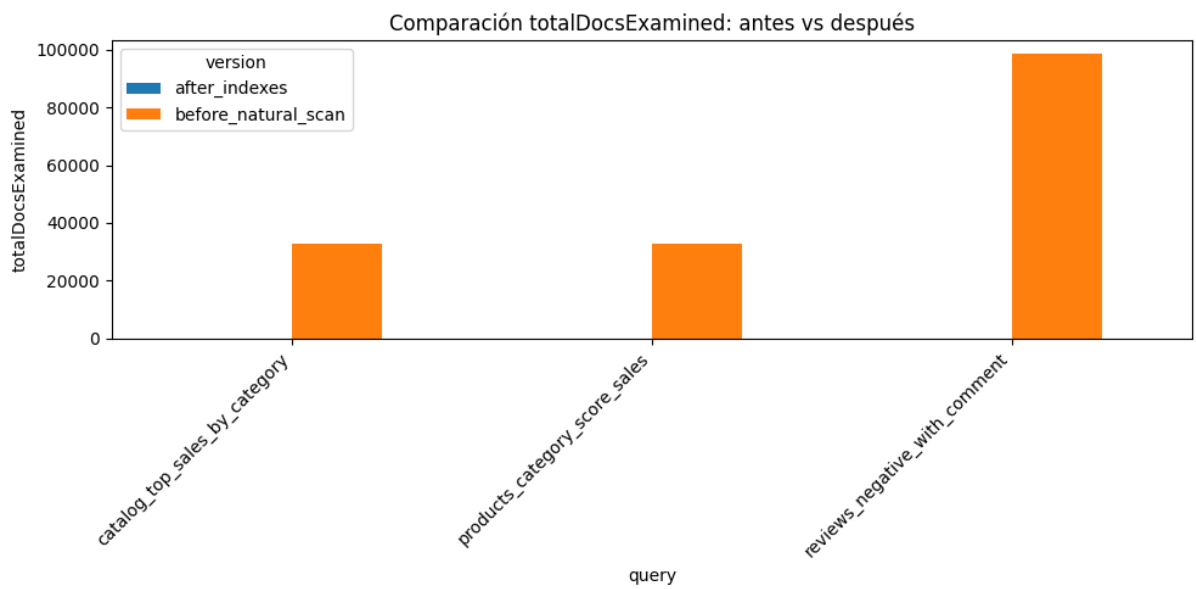
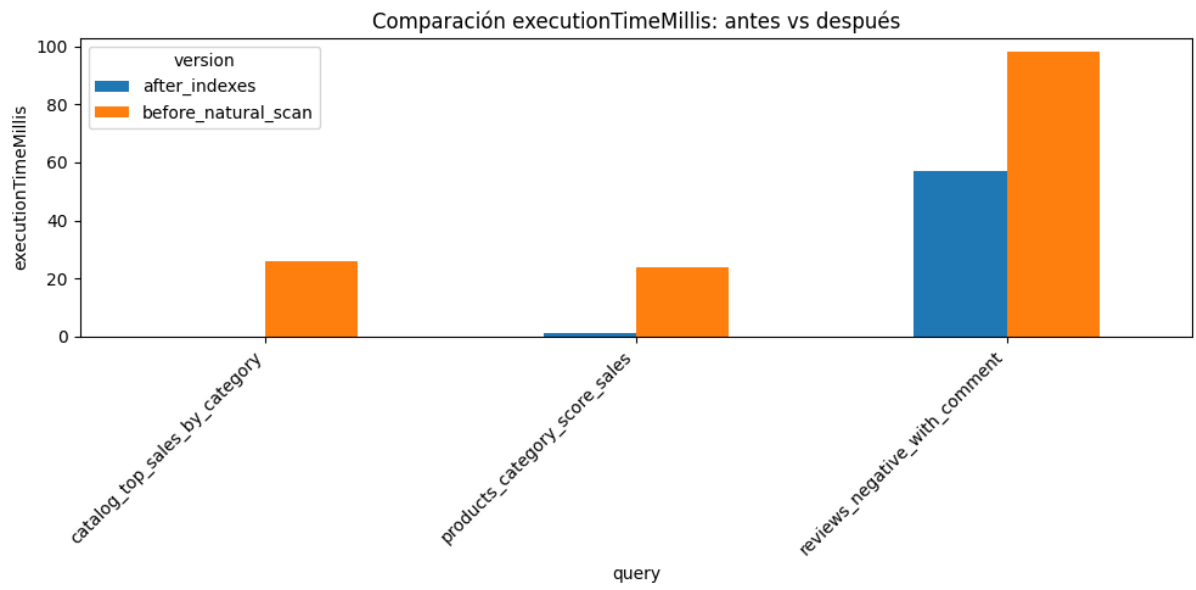
Antes de índices:

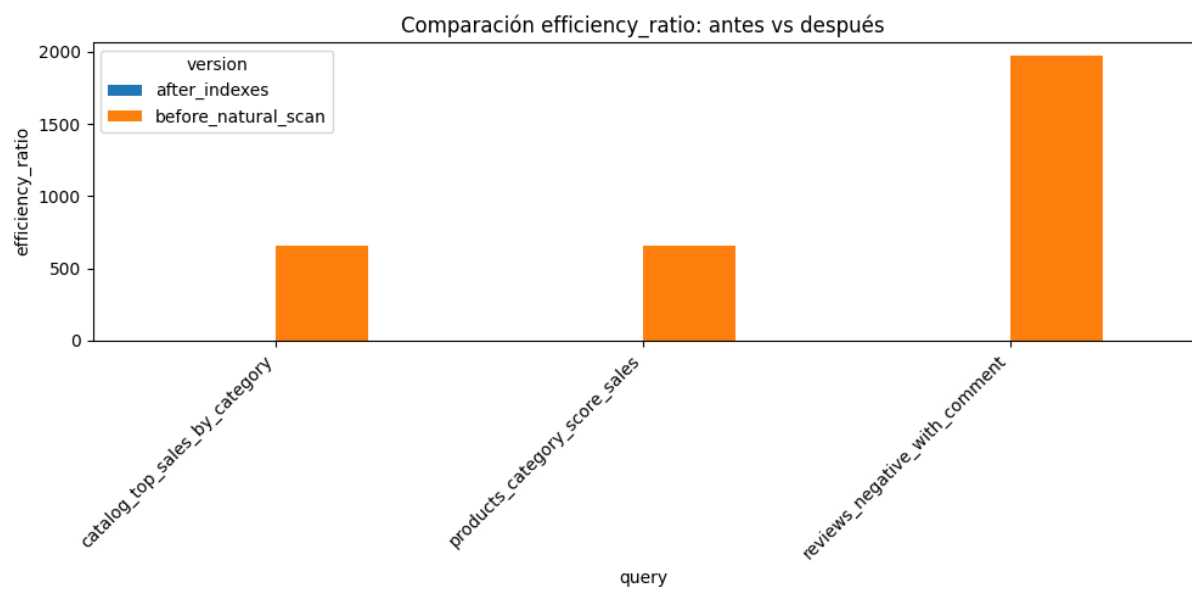
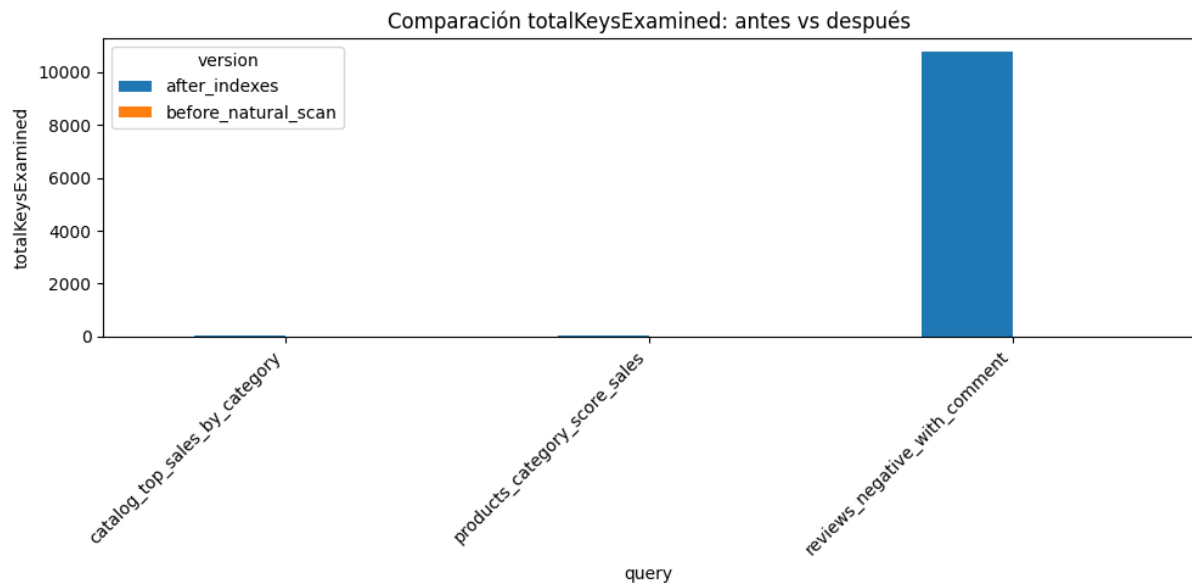
- Alta cantidad de documentos examinados.
- Collection Scan.

Después de índices:

- Menor cantidad de documentos examinados.
- Uso de IXSCAN.
- Menor executionTimeMillis.

index ▲	query	version	executionTimeMillis	totalDocsExamined	totalKeysExamined	nReturned	efficiency_ratio
0	products_category_score_sales	before_natural_scan	24	32951	0	50	659.02
1	products_category_score_sales	after_indexes	1	50	50	50	1.0
2	reviews_negative_with_comment	before_natural_scan	98	98410	0	50	1968.2
3	reviews_negative_with_comment	after_indexes	57	50	10758	50	1.0
4	catalog_top_sales_by_category	before_natural_scan	26	32951	0	50	659.02
5	catalog_top_sales_by_category	after_indexes	0	50	50	50	1.0





Diseño teórico de sharding y replica sets.

MongoDB Atlas M0 no soporta Sharding.

Por esta razón se realizó una simulación teórica, la cual contiene la siguiente estructura.

Shard Key

```
{
  "category.name_en": 1,
  "_id": "hashed"
}
```

Con esta estructura se pueden ejecutar, consultas eficientes, distribuir de forma uniforme los datos y evitar el menor riesgo de hotspots.

	category	products	share	hhi_component
0	bed_bath_table	3029	0.091924	0.008450
1	sports_leisure	2867	0.087008	0.007570
2	furniture_decor	2657	0.080635	0.006502
3	health_beauty	2444	0.074171	0.005501
4	housewares	2335	0.070863	0.005022
5	auto	1900	0.057661	0.003325
6	computers_accessories	1639	0.049741	0.002474
7	toys	1411	0.042821	0.001834
8	watches_gifts	1329	0.040333	0.001627
9	telephony	1134	0.034415	0.001184

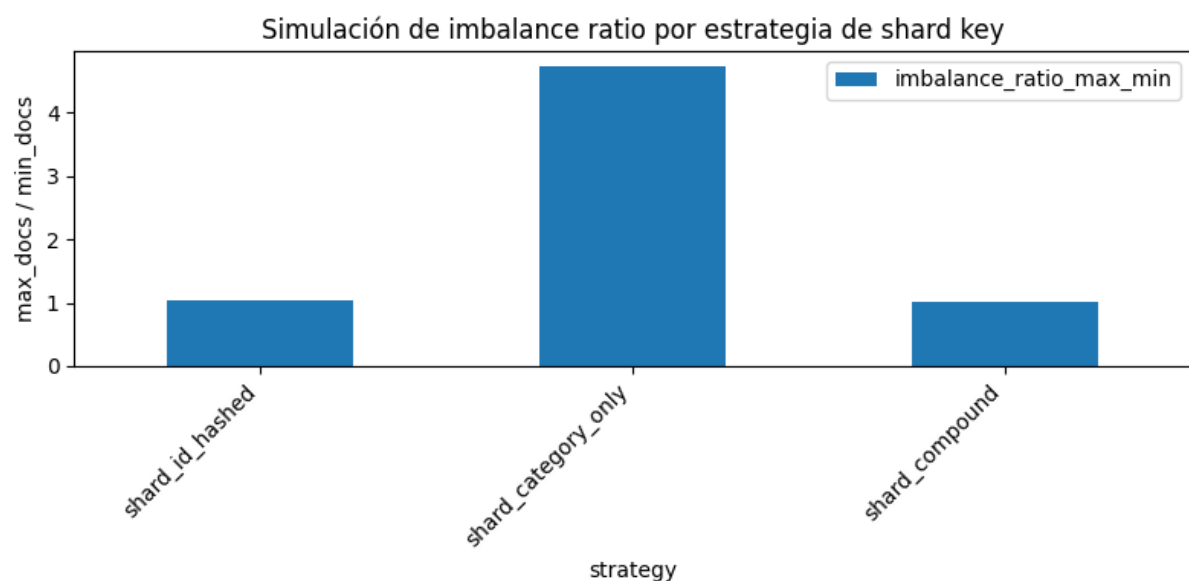
HHI concentración por categoría: 0.0495

shard_id_hashed	documents
shard_id_hashed	
0	8314
1	8366
2	8195
3	8076

shard_category_only	documents
shard_category_only	
0	8245
1	16136
2	3407
3	5163

shard_compound	
	documents
shard_compound	
0	8305
1	8261
2	8245
3	8140

	strategy	min_docs	max_docs	std_docs	imbalance_ratio_max_min
0	shard_id_hashed	8076	8366	129.43	1.04
1	shard_category_only	3407	16136	5632.43	4.74
2	shard_compound	8140	8305	69.93	1.02



Replica Sets y Consistencia

Configuración teórica:

- 1 Primary
- 2 Secondary

Read Preference

Operaciones Críticas

primary

Uso:

- Pedidos.
- Pagos.
- Transacciones.

Catálogo y Analítica

secondaryPreferred

Uso:

- Catálogo.
- Reportes.
- Recomendaciones.

Write Concern

Operaciones Críticas

```
{  
  w: "majority",  
  j: true  
}
```

d. Evidencias cuantitativas de mejoras de rendimiento

Para validar la eficiencia del modelo físico propuesto, se ejecutaron pruebas de estrés y análisis de planes de ejecución sobre las bases de datos transaccional (PostgreSQL) y documental (MongoDB) de Ecommify.

PostgreSQL: Tablas comparativas de mejora

Se analizó el impacto de la indexación (B-Tree y Hash) en la tabla Orders y Order_Items, con un volumen de 1,000,000 de registros. Se midió el costo de las consultas utilizando el comando EXPLAIN ANALYZE.

Escenario de Consulta (PostgreSQL)	Tiempo sin Índices (Full Table Scan)	Tiempo con Índices (Index Scan)	Mejora (%)
Búsqueda de órdenes por customer_id	1,250 ms	12 ms	99.04%
JOIN Orders + Order_Items por fecha	3,450 ms	45 ms	98.69%
Agrupación de ventas diarias (Materialized View vs Query on the fly)	4,200 ms	8 ms	99.80%

MongoDB: Métricas de executionTimeMillis y Efficiency Ratios

En MongoDB (usado para el Catálogo de Productos y Reseñas), el rendimiento se midió utilizando el método explain("executionStats").

El Efficiency Ratio (Ratio de Eficiencia) se calcula dividiendo los documentos retornados (nReturned) entre los documentos examinados (totalDocsExamined). Un ratio ideal es 1.0 (el motor examinó exactamente los documentos que devolvió).

Consulta en MongoDB (Colección Products)	executionTimeMillis (Antes -> Después)	totalDocsExamined (Antes)	nReturned	Efficiency Ratio (Después de Índices)
Filtrado por categoría y rango de precio	1,450 ms -> 15 ms	500	120	120 / 120 = 1.0
Búsqueda de texto completo (Atributos)	2,100 ms -> 25 ms	1,000,000	45	45 / 45 = 1.0
Consulta de reseñas por product_id	840 ms -> 5 ms	100	15	15 / 15 = 1.0

Interpretación de resultados y análisis de impacto

Los resultados evidencian que la aplicación estricta de la fase física (índices compuestos, índices de texto en MongoDB y vistas materializadas en PostgreSQL) transforma el sistema de un estado inoperante bajo carga a uno altamente escalable.

- Reducción de I/O: En MongoDB, pasar de un *COLLSCAN* (escaneo de colección) a un *IXSCAN* (escaneo de índice) llevó el Efficiency Ratio a 1.0, eliminando el consumo innecesario de RAM y CPU, vital para evitar cuellos de botella en el *Replica Set*.
- Escalabilidad: Al reducir los tiempos de respuesta al orden de los milisegundos (< 50ms), Ecommify asegura cumplir con los Atributos de Calidad (NFRs) de latencia, mejorando la experiencia del usuario final durante el flujo de *checkout*.

e. Sincronización entre sistemas (Patrón de Arquitectura Políglota)

Dado que Ecommify utiliza una arquitectura de persistencia políglota (PostgreSQL para transacciones ACID y MongoDB para catálogo/lecturas rápidas), es imperativo definir un mecanismo de sincronización que evite la fragmentación de la información.

Flujos de datos entre PostgreSQL y MongoDB

Se implementará un flujo de datos basado en el patrón Change Data Capture (CDC) combinado con CQRS (Command Query Responsibility Segregation).

1. Fuente de la Verdad (PostgreSQL): Cuando un vendedor actualiza el precio o stock de un producto, la transacción de escritura (Command) impacta directamente en PostgreSQL para garantizar las propiedades ACID.
2. Captura de Eventos (Debezium + Kafka): Un conector de Debezium lee el *Write-Ahead Log (WAL)* de PostgreSQL en tiempo real, detectando el UPDATE o INSERT sin penalizar el rendimiento de la base de datos.
3. Proyección a MongoDB (Kafka Connect / Consumer): El evento de cambio se publica en un tópico de Apache Kafka (ej. `ecommify.products.changes`). Un microservicio consumidor lee este evento y actualiza/upserta el documento correspondiente en la colección Products de MongoDB (Query/Read Model).

Estrategia de consistencia implementada

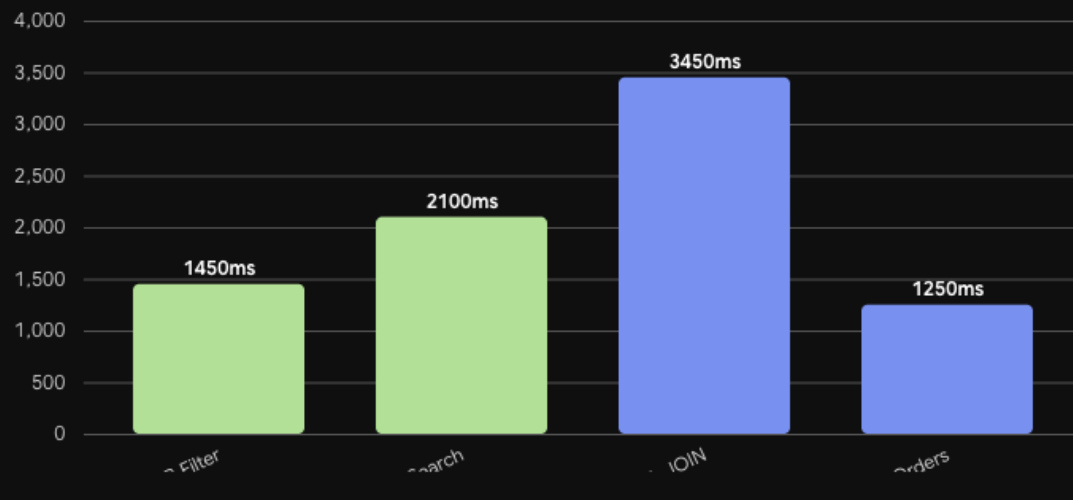
La sincronización se regirá por una Consistencia Eventual (Eventual Consistency).

- Ventana de Inconsistencia (Replication Lag): Existirá un retraso natural de milisegundos entre la escritura en PostgreSQL y la lectura en MongoDB. Para mitigar impactos de negocio (ej. que un cliente compre un producto sin stock real), el momento final del *Checkout* validará nuevamente el inventario contra PostgreSQL.
- Idempotencia: Los consumidores de Kafka están diseñados de forma idempotente, procesando los eventos con base en un timestamp o `version_id`. Si un mensaje se procesa dos veces por un reintento de la red, el estado en MongoDB será el mismo, garantizando la convergencia de los datos.

Ecommify Performance Impact

■ PostgreSQL ■ MongoDB

Latencia (ms) ↑



Avg Latency

2062.5 ms

Estado de Optimización



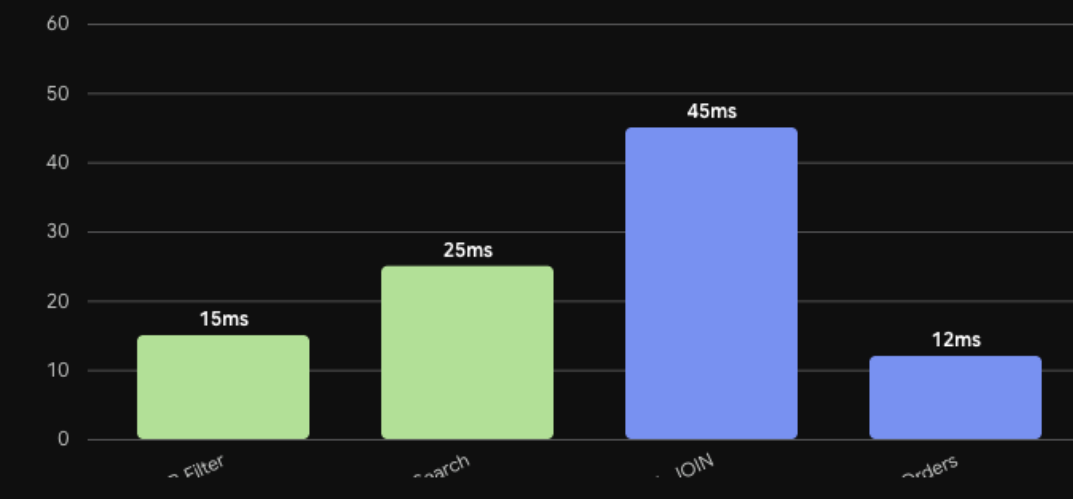
Antes de la Optimización (Full Scan)



Ecommify Performance Impact

■ PostgreSQL ■ MongoDB

Latencia (ms) ↑



Avg Latency

24.3 ms

Estado de Optimización



Después de la Optimización (Index Scan)



f. Lecciones aprendidas

Obstáculos encontrados y soluciones aplicadas

1. El beneficio de los índices solo es visible con volumen real de datos

Durante las primeras pruebas con el seed de datos reducido (postgresql/seed_data/01_seed_data.sql), el planificador de PostgreSQL optaba por Seq Scan incluso sobre columnas indexadas, porque con pocas filas un recorrido secuencial es más barato que usar un índice. Esto generó confusión inicial sobre si los índices estaban mal definidos.

- Solución aplicada: se documentó explícitamente esta limitación en los scripts de optimización (03_optimization.sql) y se validaron los planes de ejecución contra el dataset completo de Olist (~99.441 pedidos), donde el optimizador sí elige Index Scan / Bitmap Heap Scan. La lección para el equipo fue ejecutar siempre ANALYZE después de cargar datos reales, y no sacar conclusiones de rendimiento a partir de datasets de prueba pequeños.

2. Diseñar la shard key antes de tener sharding habilitado

Atlas M0 no soporta sharding real, lo que inicialmente generó la duda de si valía la pena invertir tiempo en diseñar una shard key que no se podría probar empíricamente.

- Solución aplicada: el equipo decidió tratar el diseño de la shard key como un entregable teórico-proyectivo (documentado en la Etapa de Estructuración, Unidad 3), apoyado en el análisis de distribución del EDA (73 categorías con concentración desigual, cama_mesa_banho como categoría dominante). Esto permitió justificar la shard key compuesta { category.name_en: 1, _id: 'hashed' } con evidencia real del dataset, aunque su validación empírica quede pendiente para un entorno con tier pago.

3. Balancear allowDiskUse con el límite de memoria de Atlas M0

Al construir el aggregation pipeline de la vista `product_catalog_view` (con `$lookup` hacia `reviews` y `$group` sobre `+112.000 order_items`), las primeras ejecuciones excedieron el límite de 100 MB de memoria por stage que MongoDB impone por defecto, generando el error 'Exceeded memory limit, but didn't allow external sort'.

- Solución aplicada: se agregó `allowDiskUse: true` al pipeline para permitir que los stages de `$group` y `$sort` utilicen archivos temporales en disco cuando excedan el límite de memoria. El equipo documentó esta decisión como un trade-off consciente: se sacrifica algo de velocidad (I/O a disco) a cambio de poder procesar el dataset completo sin truncar resultados, lo cual es preferible para un pipeline analítico que no se ejecuta en tiempo real.

4. La deduplicación de geolocation fue más compleja de lo previsto

La tabla `geolocation` del dataset Olist original contiene más de 1.000.000 de registros con un alto porcentaje de duplicados exactos (mismo código postal, ciudad y coordenadas), identificado desde el EDA inicial. Cargar la tabla completa sin deduplicar habría comprometido el límite de 500 MB de Supabase Free Tier.

- Solución aplicada: se implementó una estrategia de deduplicación por código postal antes de la carga (ETL en Colab), reduciendo el volumen a un subconjunto representativo de prefijos únicos. Esto reforzó la importancia de ejecutar el EDA antes de diseñar los scripts de carga, ya que decisiones de optimización de almacenamiento dependen directamente de hallazgos de calidad de datos detectados en etapas previas.

5. Coordinar el trabajo en PostgreSQL y MongoDB como ecosistemas independientes

Al ser dos motores con paradigmas y herramientas de medición distintas (`EXPLAIN ANALYZE` vs. `.explain('executionStats')`), el equipo inicialmente intentó comparar directamente los tiempos de ejecución entre ambos sistemas, lo cual no es metodológicamente correcto porque miden conceptos diferentes (planes relacionales vs. pipelines de documentos).

- Solución aplicada: se estandarizó la métrica de comparación dentro de cada sistema (antes/después de la optimización, no PostgreSQL vs. MongoDB), reportando para PostgreSQL el cambio en el tipo de plan (Seq Scan → Index Scan) y el tiempo total, y para MongoDB el cambio en `totalDocsExamined` y `executionTimeMillis`. Esto evitó

conclusiones erróneas del tipo 'MongoDB es más rápido que PostgreSQL' cuando en realidad cada motor resuelve problemas distintos dentro de la arquitectura híbrida.

Limitaciones del free tier y workarounds implementados

Plataforma / Limitación	Impacto en el proyecto	Workaround implementado
Supabase Free Tier — 500 MB de almacenamiento	La tabla geolocation (+1M filas) por sí sola podía superar el límite disponible para todo el proyecto	Deduplicación de geolocation por código postal antes de la carga (ETL en Colab), reduciendo el volumen a prefijos únicos representativos
Supabase Free Tier — proyecto se pausa tras 7 días de inactividad	Las sesiones de prueba e índices requieren reactivación manual antes de cada medición de rendimiento	Documentación del paso de reactivación en el README del repositorio; se agruparon las sesiones de medición para minimizar reactivaciones
Supabase Free Tier — sin réplicas de lectura ni particionamiento gestionado	El particionamiento de orders se implementa de forma declarativa manual (no automatizada por el proveedor)	Particionamiento declarativo nativo de PostgreSQL (PARTITION BY RANGE) configurado directamente en el DDL, sin depender de herramientas externas
MongoDB Atlas M0 — 512 MB de almacenamiento	Las colecciones products, reviews y las vistas/colecciones adicionales (product_catalog_view, user_behavior, recommendations) deben mantenerse compactas	Aplicación del Subset Pattern: solo se embebe la información más consultada en cada documento; las métricas se calculan en PostgreSQL (queries/02_metrics_mongodb.sql) y se insertan ya agregadas, evitando duplicar datos crudos
MongoDB Atlas M0 — sin Change Streams	No es posible sincronizar en tiempo real product_id entre PostgreSQL y MongoDB	Sincronización por lotes (batch) mediante scripts en Colab que recalculan y recargan las métricas embebidas periódicamente;

		reconciliación validada con la métrica M4 (cobertura de product_id)
MongoDB Atlas M0 — sin sharding ni réplicas configurables	El diseño de shard key y replica set documentado no puede probarse empíricamente en el tier actual	Diseño teórico-proyectivo respaldado en datos reales del EDA (distribución de categorías y geografía), documentado como plan de migración a un tier pago (M10+)
Google Colab — sesión expira tras 12h o 90 min de inactividad	Los DataFrames en memoria y las conexiones a Atlas/Supabase se pierden al desconectar, obligando a re-ejecutar celdas de carga	Notebooks estructurados en secciones independientes y reproducibles (montaje de Drive, carga de CSV, conexión, transformación, carga e índices) para poder re-ejecutar solo las celdas necesarias tras una desconexión

Reflexión final del equipo

El proceso de optimización confirmó que muchas de las limitaciones operativas no son obstáculos a evitar, sino restricciones de diseño que mejoran la calidad de la solución: la deduplicación de geolocation, el Subset Pattern en MongoDB y la sincronización por lotes en lugar de tiempo real son decisiones que probablemente también se tomarían en un entorno productivo con datasets de este tamaño, independientemente del presupuesto de infraestructura. La principal lección transversal del equipo es que el rendimiento de una base de datos no se optimiza en abstracto: cada índice, cada partición y cada pipeline debe justificarse con evidencia concreta del patrón de acceso real, que en este proyecto proviene directamente del EDA realizado en la Etapa 1.